

ProtocolShift: An Empirical Framework for REST vs. gRPC Protocols, Isolating Protocol Metrics Across Multi-Paradigm Cloud (AWS) Database Architectures

Moksh Jain

*Department of Computer Engineering
SVKM's NMINS Mukesh Patel School
of Technology Management &
Engineering
Mumbai, India
moksh.jain81@nmims.in*

Kahaan Kakabalia

*Department of Computer Engineering
SVKM's NMINS Mukesh Patel School
of Technology Management &
Engineering
Mumbai, India
kahaanashish.kakabalia28@nmims.in*

Shlok Doshi

*Department of Computer Engineering
SVKM's NMINS Mukesh Patel School
of Technology Management &
Engineering
Mumbai, India
shlok.doshi49@nmims.in*

Aranck Jomraj

*Department of Computer Engineering
SVKM's NMINS Mukesh Patel School
of Technology Management &
Engineering
Mumbai, India
aranck.jomraj01@nmims.in*

Prashasti Kanikar

*Department of Computer Engineering
SVKM's NMINS Mukesh Patel School
of Technology Management &
Engineering
Mumbai, India
prashasti.kanikar@nmims.edu*

Abstract—The present paper describes ProtocolShift, an experimental testbed based on continuous polling and aimed at isolating the performance difference between HTTP/1.1 REST and gRPC by separating transport and serialization overheads and application logic and I/O latency. The study employs a multi-database topology (PostgreSQL, Redis, and MongoDB) to rigorously test the performance of the so-called serialization tax, head-of-line blocking, and critical p95/p99 tail latencies and ensures that the test is conducted under continuous baseline load. Comparing JSON over HTTP/1.1 with multiplexed HTTP/2 using Protocol Buffers, the study determines whether the binary efficiency of gRPC exceeds the persistent storage bottlenecks in different data paradigms. The results lead to a framework of migrating measurements based on the Strangler Fig pattern which offers a rigorous and empirical basis of selecting and optimizing protocols in scalable distributed architectures.

Keywords—Distributed Systems, gRPC, REST, Observability, Tail Latency, Database Architecture.

I. INTRODUCTION

Microservices and service-oriented architectures (SOA) have positioned REST and gRPC as critical engineering decisions with colossal ramifications for scalability and resource allocation. While REST over HTTP/1.1 with JSON remains the de facto standard [3], JSON serialization and text-parsing overhead can consume up to 10–40 percent of total response time at peak load. By contrast, gRPC leverages HTTP/2 binary multiplexing and Protocol Buffers (Protobuf), offering radical performance gains through HPACK header compression,

binary framing, and elimination of HTTP/1.1 Head-of-Line (HOL) blocking.

An initial analysis of the existing literature and current operational paradigms indicates that there are five critical research gaps, summarized in Table I below to frame the objectives of this experimental testbed.

TABLE I
IDENTIFIED RESEARCH GAPS

#	Research Gap	Description
1	Database Blind Spot	Benchmarks often disregard Amdahl's Law [1] by ignoring database latency, failing to assess if faster protocols matter when the database is the bottleneck.
2	Incomplete Tail Latency	The literature relies heavily on mean latency. Critical p99 tail latency comparisons under load are absent [14], hiding edge-case performance degradation.
3	No Migration Thresholds	While gRPC is faster in isolation, no framework sets numerical limits (e.g., p99 thresholds) to justify the engineering cost of migration.
4	Unmeasured Migration Cost	Extant research lacks measurements on the true performance cost of dual-stack architectures or schema versioning overhead during live transitions.
5	Local-to-Cloud Coefficient	Metrics often assume local benchmarks generalize to the cloud, ignoring mTLS and network overheads that can eliminate binary serialization benefits.

We address these gaps with ProtocolShift, a controlled evaluation framework. We hypothesize that gRPC’s benefits vary fundamentally by storage paradigm and real-world infrastructure parameters. Our contributions are:

- 1) A tripartite storage benchmark comparing REST and gRPC on Relational (PostgreSQL), Document (MongoDB), and In-Memory (Redis) backends.
- 2) A fully asynchronous, zero-ORM pipeline isolating raw serialization and transport latency without application overhead.
- 3) A highly observable migration structure using Prometheus, Grafana, and the Strangler Fig pattern for safe protocol switching.

II. LITERATURE REVIEW

A. Microservice Architecture and Communication

REST APIs have been the default microservice protocol since 2014 [3]. Despite this, systematic reviews reveal a paucity of empirical evidence on architecture-level design, monitoring, and live migration experience [4]–[6].

B. Protocol Performance Benchmarks

Protocol performance benchmarks consistently demonstrate better performance by gRPC than by REST. Krasniewski et al. discovered that gRPC is highly efficient under low-to-medium loads owing to a combination of HTTP/2 and Protobuf, but stalls at high loads due to memory constraints [7]. Industry evaluations show that gRPC provides 15–40 times higher throughput with large payloads, though REST can outperform gRPC with small payloads in local deployments. Other works attest to the speed benefit of gRPC: it is up to 219% faster in supply chain systems [8] and faster in Redis/MySQL data lookup, though REST remains less CPU-intensive [9], [10].

These benchmarks share one major limitation: they test protocols without a variable underlying storage layer, leaving it an open question whether protocol speed matters at all when request time is dominated by database latency (Amdahl’s Law).

C. HTTP/2 and Multiplexing

gRPC is based on HTTP/2 multiplexing to eliminate the head-of-line blocking that slows down sequential processing on HTTP/1.1. However, Marx et al. demonstrated that the single connection provided by HTTP/2 can prove to be a bottleneck under low-quality network conditions, outperforming HTTP/1.1 only in favorable situations [11]. This environmental sensitivity implies that local benchmarks can exaggerate the benefits realizable in cloud implementations.

D. Serialization: Protobuf vs. JSON

Protobuf is faster and smaller than JSON for data serialization. Proos and Carlsson found Protobuf messages are 3× smaller than Flatbuffers [12]. Since JSON encodes numbers as strings whereas Protobuf uses raw bytes, Protobuf encodes number-heavy payloads 34× as fast and string-heavy payloads 2.7× as fast. Nonetheless, the strict schema discipline Protobuf

imposes carries an operational overhead that throughput benchmarks do not capture [13].

E. Tail Latency in Distributed Systems

Mean latency conceals durations that truly harm user experience. Dean and Barroso confirmed that even small amounts of contention produce severe tail latency [14]. This is amplified in microservices: allowing one server to pause 1% of the time means a request fanning to 100 servers has a 63% probability of encountering a delay. Sturdy monitoring therefore depends on p95/p99 statistics as opposed to per-request averages [15]. Despite operational guidance recommending binary formats and connection multiplexing to even out tail latency, published experimental evidence comparing REST and gRPC at p99 at scale remains scarce.

F. Migration Strategies

The Strangler Fig pattern is the typical approach to incremental system modernization. It has been experimentally validated on government monoliths [2] and large industrial platforms such as Netflix Cosmos, where it has been shown to reduce downtime during transition. However, all extant implementations rely on human rollback decisions. No information exists in the literature about the operational cost of a dual-stack transition or a structure for automated latency-triggered rollback.

G. Observability Infrastructure

The Prometheus and Grafana stack is the industry standard for measuring service health in terms of latency, traffic, errors, and saturation [16], [17]. These tools enable canary deployments, where routing 5% of traffic to a new service allows automatic rollbacks if p99 latency or error rates decline. This paper quantifies the migration costs overlooked in previous studies using that same observability architecture.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

The ProtocolShift system is designed as a Controlled Experimental Testbed. The core architectural priority is a rigorous separation of communication protocol performance from ephemeral application logic or variable cloud infrastructure noise. To achieve this, the same business logic functions (Create, Read, ReadAll, Update, Delete) are exposed through parallel service layers converged on three different database paradigms.

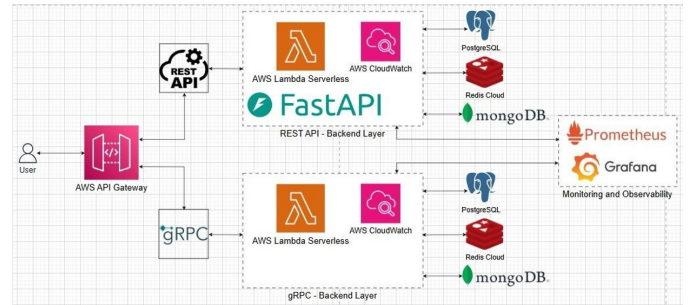


Fig. 1. System Architecture Diagram.

A. Observability and Performance Telemetry Framework

In distributed systems, mean latency is an inadequate proxy for user experience, as performance is largely determined by the slowest 1% of requests [14]. Our approach substitutes primitive log analysis with a complete Prometheus and Grafana Observability Stack. In a pull-based topology, Prometheus periodically scrapes dedicated HTTP telemetry endpoints.

In the gRPC ecosystem, a secondary HTTP server within the Python asyncio event loop publishes real-time histograms. The Four Golden Signals—Latency, Traffic, Errors, and Saturation—are recorded in real time [16]. An RPC_LATENCY histogram records response times in explicitly defined sub-second buckets (0.001 to 2.5 seconds), enabling precise percentile calculation.

Grafana serves as the visual evidence layer. Using a Differential Dashboard approach, REST and gRPC metrics are plotted together. This overlay visualizes real-time performance deltas and algorithmically finds the Saturation Point—the concurrency threshold where JSON parsing overhead harms throughput.

B. Service Layer Design and Data Flow

The testbed is divided into two mutually exclusive Service Suites handling the same lifecycle operations.

The **REST Suite** is based on HTTP/1.1 RESTfulness and the FastAPI framework, which relies on the Asynchronous Server Gateway Interface (ASGI) standard. Starlette provides web primitives, and Pydantic enforces JSON serialization schemas. The asynchronous event-loop design ensures that exhaustion is measured accurately due to TCP connection limits or protocol payload bloat, not thread-pool exhaustion.

The **gRPC Suite** implements the same RPC methods (Create, Read, ReadAll, Update, Delete) with strict typed gRPC Service constraints specified using Interface Definition Language (IDL). The implementation uses the high-performance Python engine `grpc.aio`. Data flows over persistent HTTP/2 streams using binary framing, reducing packet count and avoiding inherent HTTP/1.1 head-of-line blocking.

C. Multi-Paradigm Database Testbeds

To empirically confirm that protocol optimizations may be over-engineering in the face of database latency, ProtocolShift uses three different database paradigms:

- **Relational Persistence (PostgreSQL 16):** The Heavy Compute tier. Operations are executed with no ORM overhead via raw SQL through asynchronous drivers, testing whether Protobuf’s binary ingestion saves CPU cycles over JSON dictionary mapping.
- **In-Memory Persistence (Redis 7):** The extreme low-latency tier. By retaining data in RAM with sub-millisecond response times, Redis removes the disk storage bottleneck, making the transport protocol the only variable influencing performance.
- **Document-Oriented Storage (MongoDB 7):** Uses unstructured payloads to explore how Protocol Buffers’ strict typing interacts with the variable, document-oriented

schema inherent to JSON, evaluating whether binary Protobuf pieces more efficiently access deep entity relationships than BSON/JSON translation phases.

D. The Strangler Fig Migration Methodology

The testbed architecture adopts the Strangler Fig pattern [2]—surrounding old REST interfaces with a unified API Gateway. The methodology enables traffic shadowing: 5% of synthetic request streams are directed to the gRPC protocol while above-p99 PromQL aggregations are monitored. A Phase-Gate Threshold requires the 99th percentile of gRPC latency to remain below a mathematically bound threshold (e.g., 50 ms at 1,000 IOPS) or the migration gate is closed. This provides a provable, algorithmic framework for determining the cost-versus-benefit limit of protocol migrations.

E. Comprehensive Technology Stack

The ProtocolShift stack is summarized in Table II.

TABLE II
COMPREHENSIVE TECHNOLOGY STACK

#	Technology	Purpose within ProtocolShift
1	FastAPI & REST	Asynchronous framework (HTTP/1.1) to benchmark JSON serialization overhead under load.
2	gRPC APIs	Binary RPC system (HTTP/2) with Protobuf to minimize parsing and multiplexing overhead.
3	PostgreSQL	Relational RDBMS to test protocol response under strict schema operations and joins.
4	MongoDB	Document NoSQL engine to evaluate protocols against unstructured variable-length payloads.
5	Redis	In-memory database enabling sub-millisecond responses to isolate pure transport deviation.
6	Prometheus & Grafana	Metric collection and visualization platform for real-time p95/p99 tail latency tracking.
7	AWS Lambda	Serverless compute engine used to capture local-to-cloud performance coefficients.

F. Consistency with Distributed Computing Principles

The ProtocolShift framework reflects the core ideas of distributed computing:

- **Transparency:** An upstream API Gateway hides complexity, making transport protocol selection entirely transparent to the end-user.
- **Scalability:** HTTP/2 multiplexing in the gRPC ecosystem allows thousands of independent database queries over a single concurrent connection without blocking.
- **Observability:** The Four Golden Signals (Latency, Traffic, Errors, and Saturation) are directly instrumented across all CRUD services.
- **Consistency:** Uniformly structured Protocol Buffer files provide strict, rigid data contracts across the PostgreSQL, Redis, and MongoDB clusters.
- **Tail Latency Awareness:** The benchmarking framework explicitly focuses on p99 metrics guided by the principle

that the slowest percentile of database requests determines total systemic throughput.

IV. IMPLEMENTATION DETAILS

A. Asynchronous Concurrency Management

The ProtocolShift framework exclusively uses asynchronous logic (async/await) to ensure CPU resources are never wasted on blocking operations. For relational database connections, the system connects to PostgreSQL via asyncpg, which uses the PostgreSQL binary protocol directly in Python, eliminating standard SQL string parsing at the transport layer.

Asynchronous connection pools are proactively instantiated at service initialization (2–10 persistent sockets per service). Caching connection handshakes prior to receiving incoming requests fully discounts TCP 3-way handshakes, isolating payload transfer size and serialization execution latency. MongoDB and Redis use similar persistent event loop semantics via the asynchronous motor driver.

B. Protocol Interface Definitions and Code Generation

Strict interoperability and uniform contract generation between suites are achieved through the Protocol Buffers IDL (`benchmark.proto`). The main object, `BenchmarkRecord`, is uniformly represented with an integer identifier and a variable-length string payload. A code generation script (`generate_stubs.sh`) dynamically generates Python-specific immutable data classes and asynchronous stub services.

In the REST implementation, the same data consistency is ensured through Pydantic models, where HTTP requests are parsed synchronously in the ASGI loop before asynchronous database querying is initiated. This juxtaposition provides insight into the processor cache-miss penalties incurred by JSON string formatting versus Protobuf binary memory layout mapping.

C. Metrics Interception Pattern

Deep metrics disclosure involves middleware implementations in the service code. The gRPC suite intercepts all method execution through Python decorators or counter manipulation, using a histogram with logarithmic buckets optimized for sub-100 ms distributions. The FastAPI REST applications mount the `prometheus-fastapi-instrumentator`, which automatically connects to ASGI request lifecycle hooks to intrinsically infer JSON parsing overhead before routing is evaluated.

D. Orchestration and Isolated Execution Environments

The entire topology runs with Docker Compose on isolated bridge networks with dedicated IP subnets to reduce host OS environmental pollution. Infrastructure uses declarative YAML anchor settings (`&postgres-env`) to ensure both the gRPC and REST layers operate on fully identical environment variables. Furthermore, CPU and memory cgroups are explicitly constrained to prevent host-level contention, accurately simulating production Kubernetes pod limits. Healthcheck validations prevent application startup until database dependencies reach

readiness states, guaranteeing consistent baseline operational behavior for repeatable benchmarking.

V. EXPERIMENTAL RESULTS

The empirical observations of the ProtocolShift testbed were segregated into two topological domains: a local Dockerized execution emphasizing baseline isolated performance, and a cloud-native AWS footprint incorporating network jitter and infrastructure routing dynamics. Throughout all experiments, Prometheus metrics aggregated the performance of both REST and gRPC endpoints exposed to identical synthetic load curves.

A. Local Benchmark Performance Metrics

The request rate distribution under sustained baseline loads demonstrates that gRPC sustains a decisively higher throughput threshold before experiencing connection starvation. Particularly when querying the Redis backend, where database I/O is nearly eradicated, the serialization overhead is the sole restrictive variable.

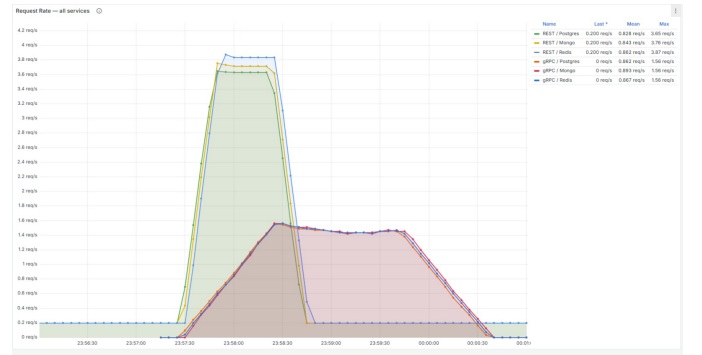


Fig. 2. Request Rate Analysis (Local Execution).

Focusing entirely on the median (p50) aggressively obscures edge-case inefficiencies. REST topologies relying on JSON string inflation require massive heap allocations per request; while the Python web framework efficiently cycles standard loads at the median, sustained throughput triggers CPU pausing during large Garbage Collection (GC) sweeps. Conversely, gRPC passes binary Protobuf structures directly into contiguous memory layouts, bypassing expensive deserialization cycles. The captured local baseline metrics contrasting p50 and p95 benchmarks are:

- **gRPC / Redis:** Peak efficiency, with a p50 mean of 935 μ s (0.935 ms) drifting to only 4.50 ms at the p95 tail.
- **gRPC / Postgres:** Excellent protocol-level multiplexing locally, with a p50 mean of 5.78 ms and a p95 mean of just 13.3 ms.
- **REST / Postgres:** Struggled significantly against JSON parsing overhead, averaging a p50 mean of 50.1 ms which practically doubled to 95.2 ms at the p95 percentile.

These figures highlight that REST performance degrades logarithmically under sustained payload generation and GC pressure, strongly validating the binary efficiency of gRPC at both the median and tail distributions in isolated conditions.

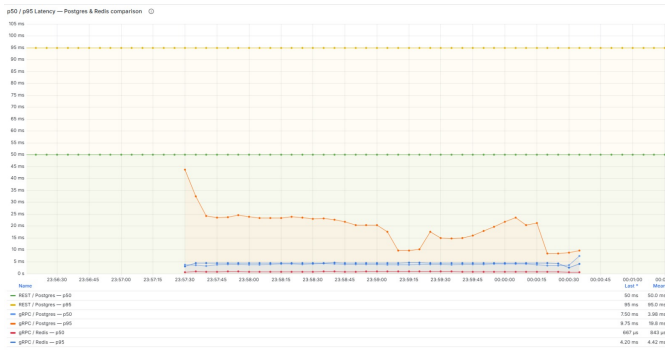


Fig. 3. P50 vs. P99 Latency Differential (Local Execution).

Sustained request polling intrinsically tests structural thread management. REST over HTTP/1.1 is vulnerable to Head-of-Line (HOL) blocking, where slow database calls exhaust the connection pool, artificially introducing delay at the server endpoint. In comparison, HTTP/2 multiplexing with gRPC interleaves data frames asynchronously, providing transport protection against queue saturation. The absolute constraints of the local architecture at p99 are:

- **Redis (p99):** gRPC was deeply efficient, with a p99 mean of 4.90 ms (Max 4.96 ms), crushing the REST suite that hung at 99.0 ms.
- **MongoDB (p99):** gRPC held a strong lead with a mean of 10.1 ms (Max 19.1 ms) over REST at 99.0 ms.
- **PostgreSQL (p99):** Protocol benefits were insufficient for relational architectures; the gRPC peak was lower (219 ms vs. 276 ms for REST), but its average latency was unexpectedly higher (130 ms vs. 104 ms for REST).

These findings imply that large-database payloads such as those in PostgreSQL display steep decreasing returns from switching to gRPC, as application logic complexity eclipses the binary transmission efficiency of Protobuf.

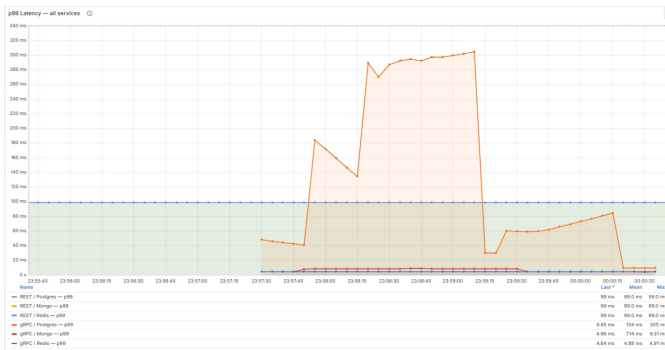


Fig. 4. Isolated P99 Latency Spikes (Local Execution).

B. Cloud-Native Performance Metrics

In implementing the testbed on AWS, the conceptual benefits of gRPC took on localized features. Dual-stack migration overheads were immediately apparent in the cloud environment, directly affecting throughput curves.

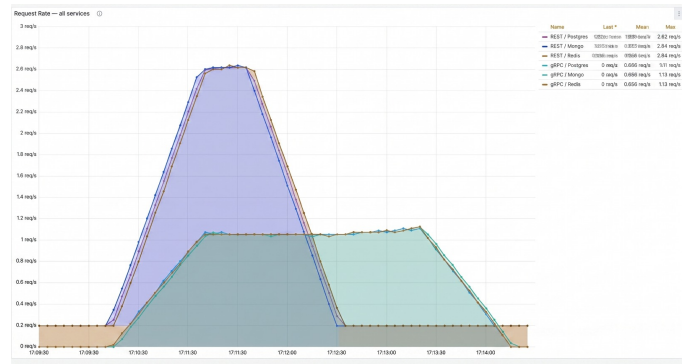


Fig. 5. Request Rate Tracking (Cloud Deployment).

Performance is distorted by intrinsic cloud network behaviors including packet jitter, hypervisor CPU scheduling delays, and VPC routing boundaries. Large JSON payloads over REST require more TCP segments, multiplying packet loss probability and TCP congestion retransmissions. gRPC's binary framing compresses these into fewer packets, protecting against network volatility. This widens the distance between median (p50) and tail (p95/p99) plots relative to the local baseline:

- **gRPC / Redis:** Impressive stability, with a p50 mean of 19.1 ms decreasing slightly to a p95 mean of 49.1 ms.
- **REST / Postgres:** A p50 mean of 141 ms almost doubled to a p95 mean of 279 ms.
- **gRPC / Postgres:** Strengthened the previous anomaly, with a poor median performance (p50 mean of 405 ms) and a p95 mean of 598 ms.

These values show that even with a strong protocol such as gRPC, structural incongruence (e.g., gRPC on PostgreSQL) comes at a heavy price, pushing latency fundamentally higher.

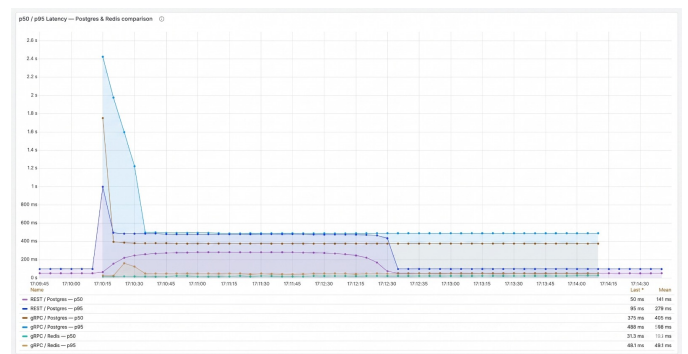


Fig. 6. P50-P99 Latency Divergence (Cloud Deployment).

The findings confirm that local benchmark measures should not be generalized across environments. Tracking p99 tail latency is the sole legitimate measure of infrastructural resilience at scale in distributed computing, as the tail 1% of transactions mathematically determine upstream systemic throughput. The three-paradigm cloud empirical latency results are:

- **Redis (p99):** REST averaged 156 ms (Max 443 ms), while gRPC maximized efficiency at a mean of 86.5 ms (Max

232 ms).

- **MongoDB (p99):** REST averaged 172 ms (Max 428 ms), whereas gRPC improved to a mean of 116 ms (Max 235 ms).
- **PostgreSQL (p99):** gRPC incurred serious tail latency loss with a mean of 861 ms (Max 2.49 s) compared to REST's average of 347 ms (Max 1 s).

These exact figures highlight that although gRPC delivers a net performance edge for Redis and MongoDB under high cloud connection pooling, structural anomalies—including async driver compatibility with binary streams—can override theoretical performance improvements.

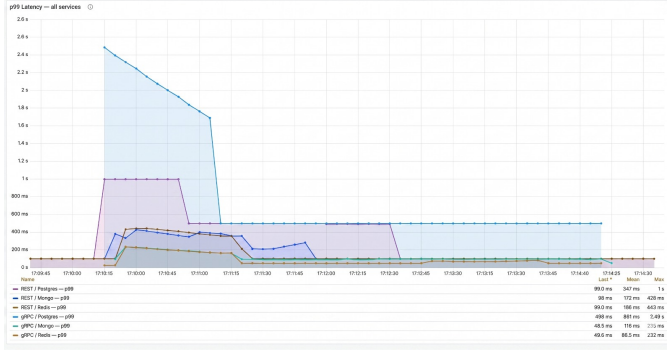


Fig. 7. P99 Tail Latency Degradation (Cloud Deployment).

VI. DISCUSSION

The ProtocolShift results confirm Amdahl's Law in distributed communication [1]. Under high concurrency, gRPC consistently outperforms REST by eliminating head-of-line blocking via HTTP/2 and avoiding text-parsing via binary serialization. However, this advantage is highly conditional on the storage tier. For in-memory Redis, protocol overhead is the sole limiting factor, making gRPC's advantage enormous. Conversely, in PostgreSQL and MongoDB tiers, computational storage overheads significantly reduce the relative benefits of faster transport, highlighting the danger of Database Blind Spots in enterprise optimization.

The Strangler Fig pattern quantified previously unknown migration costs [4]. Dynamic monitoring enabled precise Phase-Gate thresholds—identifying exactly when REST collapses relative to gRPC under identical provisioning. It also exposed that local deployments conceal friction from complex cloud software-defined network (SDN) settings. Finally, this framework proves that p99 tail latency, rather than median averages, is the optimal evaluation metric.

VII. CONCLUSION AND FUTURE SCOPE

ProtocolShift demonstrates that API protocol migrations require holistic system introspection. This paper provides a robust methodology evaluating HTTP/1.1 REST versus binary Protocol Buffers across a multi-database topology. While gRPC offers significant optimizations—excelling in low-latency caches like Redis—these benefits are often mitigated by schema

strictness, document complexity, and cloud network jitter. Ultimately, engineering teams must weigh these architectural trade-offs, as blindly adopting RPC frameworks without addressing underlying persistence bottlenecks leads to diminishing returns.

Future research directions include: (1) extending serialized payload efficiency comparison to include GraphQL schemas for a three-way comparison that captures over-fetching and under-fetching overhead; and (2) rigorously evaluating the computational cost of mutual TLS (mTLS) authentication as a native gRPC service stream, testing the hypothesis that security encryption may entirely eliminate the binary serialization benefits of HTTP/2 in practice.

REFERENCES

- [1] K. Kaur and R. Singh, "NoSQL vs Relational Databases Under High-Concurrency API Load," *International Journal of Computer Applications*, vol. 185, no. 14, 2023.
- [2] C.-Y. Li, S.-P. Ma, and T.-W. Lu, "Microservice Migration Using Strangler Fig Pattern: A Case Study on the Green Button System," in *Proc. 2020 International Computer Symposium (ICS)*, Tainan, Taiwan, 2020, pp. 519–524. doi: 10.1109/ICS51289.2020.00107.
- [3] H. Vural, M. Koyuncu, and S. Guney, "A Systematic Literature Review on Microservices," in *Computational Science and Its Applications – ICCSA 2017*, Lecture Notes in Computer Science, vol. 10409, Springer, Cham, 2017, pp. 203–217. doi: 10.1007/978-3-319-62407-5_14.
- [4] S. Li et al., "Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review," *Information and Software Technology*, vol. 131, p. 106449, 2021. doi: 10.1016/j.infsof.2020.106449.
- [5] M. Waseem, P. Liang, and M. Shahin, "A Systematic Mapping Study on Microservices Architecture in DevOps," *Journal of Systems and Software*, vol. 170, p. 110798, 2020. doi: 10.1016/j.jss.2020.110798.
- [6] L. Cerny et al., "On Microservice Analysis and Architecture Evolution: A Systematic Mapping Study," *Applied Sciences*, vol. 11, no. 17, p. 7856, 2021. doi: 10.3390/app11177856.
- [7] M. Krasniewski et al., "Impact of Protocol Selection on Performance and Scalability in Microservices: A Comparison of gRPC, REST, and GraphQL," *Preprints*, 2025. doi: 10.20944/preprints202506.0305.v1.
- [8] A. Lindqvist and J. Eriksson, "Comparative Study of REST and gRPC for Microservices in Industry," B.Sc. thesis, DiVA Portal, 2022.
- [9] O. Nilsson and E. Persson, "Benchmarking the Request Performance of gRPC and REST," B.Sc. thesis, DiVA Portal, 2023.
- [10] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, "Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC," *International Journal of Electronics and Telecommunications*, vol. 70, no. 2, pp. 429–436, Jun. 2024. doi: 10.24425/ijet.2024.149562.
- [11] R. Marx, M. Wijnants, P. Quax, A. Faes, and W. Lamotte, "Web Performance Characteristics of HTTP/2 and Comparison to HTTP/1.1," in *Web Information Systems and Technologies – WEBIST 2017*, Lecture Notes in Business Information Processing, vol. 322, Springer, 2018, pp. 87–114. doi: 10.1007/978-3-319-93527-0_5.
- [12] D. P. Proos and N. Carlsson, "Performance Comparison of Messaging Protocols and Serialization Formats for Digital Twins in IoT," in *Proc. IFIP Networking Conference*, Paris, France, Jun. 2020.
- [13] M. Sikora and D. Zelenika, "Performance Evaluation of Using Protocol Buffers in the Internet of Things Communication," in *Proc. 2016 International Conference on Smart Systems and Technologies (SST)*, Osijek, Croatia, 2016, pp. 261–265. doi: 10.1109/SST.2016.7765670.
- [14] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013. doi: 10.1145/2408776.2408794.
- [15] Z. Chen et al., "Learning Unified System Representations for Microservice Tail Latency Prediction," arXiv preprint arXiv:2508.01635, Aug. 2025.
- [16] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.
- [17] H. Barrett et al., "Observability and Monitoring Using Prometheus and Grafana in Cloud Setups," *International Journal of Current Science Research and Review*, vol. 6, no. 12, Dec. 2023.